

Memory Analysis Report

StaySuite-HospitalityOS

Comprehensive memory consumption audit, root cause analysis,
and prioritized optimization roadmap

Project	StaySuite-HospitalityOS
Stack	Next.js 16.2.4 + Turbopack + React 19 + Prisma 6 + PostgreSQL
Server	4 vCPU / 8 GB RAM / Ubuntu Linux / Node.js 24.15.0
Source Files	1,870 TypeScript files (918 API routes, 187 pages)
Dependencies	107 packages (1.5 GB node_modules)
Dev Build Cache	2.6 GB (.next/dev/) with 2,599 JS chunks
Date	May 20, 2026

1. Executive Summary

The StaySuite-HospitalityOS application is running on an 8 GB RAM server with 4 vCPUs. The system has experienced repeated Out-Of-Memory (OOM) kills by the Linux kernel, causing the Next.js dev server to crash. At the time of analysis, the system had only 508 MB free memory and 767 MB in buffer/cache, with 6.9 GB actively used.

OOM Kill Evidence (dmesg): Kernel killed next-server process (PID 22587) with total-vm: 31,402 MB, anon-rss: 7,026 MB. A second kill (PID 31369) showed total-vm: 31,168 MB, anon-rss: 6,546 MB. The Turbopack dependency tracer consumed 4-5 GB during module analysis before being killed.

Key Finding: The primary OOM root cause is Turbopack static analysis of the scheduler dependency chain in instrumentation.ts. This single import chain pulls in crypto, net, child_process, twilio, node-cron, and the entire WiFi adapter tree, consuming 4-5 GB of memory during compilation. This has been partially mitigated by moving the scheduler to a separate PM2 process, but the 2.6 GB Turbopack dev cache and 1.5 GB node_modules continue to strain the 8 GB server.

2. System Resource Overview

2.1 Current Memory State

Metric	Value	Notes
Total RAM	8,082 MB	Hard limit of the VM/container
Used	6,900 MB (85%)	Critically high - near OOM threshold
Free	508 MB (6%)	Insufficient for compilation spikes
Buffer/Cache	767 MB (10%)	Can be reclaimed under pressure
Swap	0 MB	No swap configured - no safety net
Next.js Process	77.7 MB	Currently running (post-OOM restart)
Scheduler Process	44.9 MB	Running separately via PM2
PM2 Restarts	9 restarts	Indicates repeated OOM recovery
Max Memory Limit	3 GB	PM2 max_memory_restart threshold

Table 1: System memory state at time of analysis

2.2 Disk Usage Breakdown

Directory	Size	Description
node_modules/	1.5 GB	107 packages including heavy deps (twilio, pg, ldapjs)

.next/dev/	2.6 GB	Turbopack dev cache - 2,599 compiled JS chunks (96 MB total)
.next/dev/static/chunks/	283 MB	Client-side compiled bundles
.next/dev/server/	404 MB	Server-side compiled chunks
src/	42 MB	1,870 TypeScript source files
public/images/	2.3 MB	Login slides and logo images

Table 2: Project disk usage breakdown

3. Root Cause Analysis

3.1 Primary Cause: Turbopack Tracing Scheduler Dependencies

The root cause of the OOM kills was identified in a previous session. The `instrumentation.ts` file (which Next.js loads at server startup) previously imported the scheduler, which transitively pulled in the entire WiFi adapter tree, crypto, net, child_process, twilio, node-cron, and other heavy native modules. Turbopack statically analyzes ALL imports, even dynamic ones inside `setTimeout` blocks, and traces the full dependency graph. This consumed 4-5 GB of memory during compilation.

Dependency Chain (pre-fix):

```
instrumentation.ts
+-- scheduler.ts (node-cron, encryption)
+-- crypto (native binding, 3-5 MB)
+-- wifi/adapters/index.ts (15 vendor adapters)
+-- net (native binding, lazy but TRACED)
+-- session-engine.ts
+-- child_process (native binding)
+-- nftables-counters.ts
+-- child_process (native binding)
+-- nas-health-check.ts
+-- child_process + net + dgram (3 native bindings)
+-- db.ts
+-- PrismaClient (20-40 MB per instance)
+-- pg connection pool
+-- auth.ts
+-- next-auth (10-20 MB)
+-- bcryptjs (2 MB with WASM)
```

3.2 Current Mitigation: PM2 Dual-Process Architecture

The scheduler has been removed from `instrumentation.ts` and now runs as a separate PM2-managed process. This prevents Turbopack from tracing the scheduler dependency chain. The current architecture consists of two processes:

Process	Command	Max Memory	Port	Restarts
---------	---------	------------	------	----------

staysuite-nextjs	npx next dev --turbo	3 GB	Port 3000	9 restarts
staysuite-scheduler	npx tsx scripts/scheduler-runner.ts	1.5 GB	N/A	0 restarts

Table 3: PM2 process configuration

4. Ongoing Memory Pressure Points

4.1 Oversized Compiled Chunks (20 files over 1 MB)

The Turbopack dev compiler generates 2,599 JavaScript chunks totaling 96 MB. Of these, 20 individual files exceed 1 MB each. The largest chunks contain dashboard widgets, WiFi portal pages, and heavy dependencies like hls.js (video streaming) and recharts.

Chunk	Size	Contents
dashboard widgets (client)	2.5 MB	framer-motion + recharts + state
dashboard widgets (server SSR)	2.4 MB	Duplicated server rendering bundle
WiFi portal page (client)	1.5 MB	Complex form + WiFi config UI
hls.js (client)	1.5 MB	Video streaming library - loaded on WiFi page
WiFi reports page (client)	1.2 MB	recharts + xlsx parser
Prisma WASM (edge)	1.4 MB	Edge runtime Prisma client
twilio rest (edge)	1.3 MB	Twilio SDK in edge chunks (should be external)
twilio account (edge)	1.2 MB	Twilio SDK sub-module in edge
root-of-server (server)	1.2 MB	Core Next.js server bundle
react-dom (client)	1.1 MB	React DOM runtime
Next.js router (client)	1.1 MB	Client-side router

Table 4: Largest compiled chunks (>1 MB each)

4.2 Client-Side Bundle Bloat

The client bundle includes several heavy libraries that are loaded eagerly, significantly increasing browser memory usage and initial page load time:

Library	Import Count	Estimated Size	Issue
framer-motion	95 files	~150 KB gzipped	Used for simple fade transitions; full library loaded
recharts	35 files	~500 KB gzipped	Heavy charting library; imported in dashboard, reports, revenue

lucide-react	554 imports (198 unique icons)	Variable (tree-shaken)	Massive import graph; 198 unique icons across codebase
socket.io-client	8 files	~80 KB gzipped	Two duplicate hooks (useSocket + useRealtime)
hls.js	1 file	~1.5 MB	Video player - loaded on WiFi portal page
jspdf + autotable	2 files	~300 KB	PDF generation - server-side only (OK)

Table 5: Client-side bundle analysis

4.3 Duplicate WebSocket Connections

Two separate hooks, `useRealtime` (653 lines) and `useSocket` (382 lines), both create independent `socket.io-client` connections. If both are mounted simultaneously, that results in 2 parallel WebSocket connections per browser tab. The `useRealtime` hook auto-connects on every authenticated page and attempts 10 reconnection attempts with toast notifications on each failure.

4.4 Context Initialization Storm

On application mount, 4 React contexts fire independent API calls simultaneously: `AuthContext` (session check), `FeatureFlagsContext`, `CurrencyContext`, and `TimezoneContext`. Additionally, `CurrencyContext` and `TaxContext` both independently fetch from the same `/api/settings/tax-currency` endpoint, creating duplicate requests.

Context	Endpoint	State Cached	Fetches on Mount
<code>AuthContext</code>	GET <code>/api/auth/session</code>	User object + permissions	Yes (session + retry)
<code>FeatureFlagsContext</code>	GET <code>/api/settings/feature-flags</code>	<code>enabledFeatures[]</code> + plan	Yes
<code>CurrencyContext</code>	GET <code>/api/settings/tax-currency</code>	<code>CurrencySettings</code>	Yes
<code>TimezoneContext</code>	GET <code>/api/settings/general</code>	<code>TimezoneSettings</code>	Yes
<code>TaxContext</code>	GET <code>/api/settings/tax-currency</code>	<code>Tax[]</code> + <code>TaxGroup[]</code>	Yes (DUPLICATE)

Table 6: Context initialization API calls

4.5 Aggressive Polling Intervals

Multiple hooks run continuous polling intervals that consume memory and network bandwidth:

Hook	Interval	Method	Notes
<code>useDashboardData</code>	45 seconds	GET <code>/api/dashboard</code>	Ref-counted, deduped - well designed
<code>useLicenseCheck</code>	30 seconds	GET <code>/api/license/check</code>	Per-component instance, visibility API
<code>useRealtime</code>	Continuous	WebSocket (<code>socket.io</code>)	Auto-reconnect, 10 retries with toasts

Table 7: Active polling intervals

4.6 NodePolyfillPlugin on Client Bundle

The `next.config.ts` adds `NodePolyfillPlugin()` to the webpack client configuration, which injects polyfills for `buffer`, `process`, `stream`, and other Node.js built-ins into the client bundle. This adds approximately 200+ KB to every client-side page load for polyfills that are mostly unnecessary. The config also marks all Node.js built-ins (`crypto`, `fs`, `net`, etc.) as `false` for the client, but the polyfill plugin contradicts this by providing them anyway.

4.7 Dead Configuration in next.config.ts

Several entries in `serverExternalPackages` are unused. The packages `pg`, `pg-native`, and `twilio` are listed but never imported anywhere in the source code. Additionally, `ignoreBuildErrors: true` disables TypeScript checking to avoid OOM during build (the project has 1,870 files; `tsc` requires >2 GB). `reactStrictMode: false` disables React strict mode, which would normally help detect memory leaks from double renders.

4.8 Unbounded Tenant PrismaClient Cache

The `db-tenant.ts` creates a new `PrismaClient` instance per tenant and caches it in a `Map`. There is no LRU eviction or TTL - tenants accumulate connection pools indefinitely. Each `PrismaClient` instance consumes 20-40 MB. With many tenants, this becomes a significant memory leak risk.

5. Prioritized Solutions

5.1 Critical (Immediate Impact)

ID	Action	Details	Impact
P1	Consolidate <code>useSocket</code> + <code>useRealtime</code>	Eliminate duplicate <code>WebSocket</code> ; single hook, single connection	Reduces per-tab memory by ~80 KB and prevents connection spam
P2	Remove <code>NodePolyfillPlugin</code> from client	Delete <code>NodePolyfillPlugin()</code> from webpack config	Saves ~200 KB from every client page load
P3	Replace <code>framer-motion</code> with CSS transitions	Use CSS transitions/animations on <code>page.tsx</code> shell	Eliminates ~150 KB from initial bundle
P4	Lazy-load recharts components	Use <code>next/dynamic</code> for chart components	Defers ~500 KB until dashboard/reports pages load

Table 8: Critical priority solutions

5.2 High Priority (Significant Impact)

ID	Action	Details	Impact
P5	Merge 4 context fetches into 1 API call	Create <code>/api/settings/bundle</code> endpoint	Reduces 4 concurrent requests to 1; faster mount

P6	Add LRU eviction to tenant PrismaClient cache	Max 20 tenants, TTL 1 hour	Prevents unbounded memory growth from tenant connections
P7	Remove dead serverExternalPackages	Remove pg, pg-native, twilio from config	Cleaner config; no runtime impact but reduces confusion
P8	Lazy-import bcryptjs in auth.ts	Use await import('bcryptjs') in verify function	Defers 2 MB WASM bundle until first password check
P9	Enable reactStrictMode	Set reactStrictMode: true in config	Helps detect memory leaks from effect cleanup issues

Table 9: High priority solutions

5.3 Medium Priority (Optimization)

ID	Action	Details	Impact
P10	Add swap space to the server	Create 2 GB swap file	Provides OOM safety net; prevents hard kills
P11	Clean Turbopack cache periodically	Add cron job to clear .next/dev/ on idle	Prevents 2.6 GB cache from consuming all disk
P12	Audit and remove unused packages	Remove ldapjs, pg (if unused) from package.json	Reduces node_modules from 1.5 GB
P13	Consolidate lucide-react icon imports	Create centralized icon registry	Reduces 554 import points to cleaner pattern
P14	Lazy-load hls.js on WiFi portal only	Dynamic import when WiFi portal mounts	Defers 1.5 MB until WiFi page is visited

Table 10: Medium priority solutions

6. Memory Budget Projection

The following table estimates the memory usage before and after implementing the critical and high-priority solutions. The projection assumes typical dev-mode operation with Turbopack and multiple page compilations.

Component	Current	After Optimization	Notes
Next.js dev server (Turbopack)	1,500 MB	1,200 MB	Smaller chunks from lazy loading
Turbopack compilation spikes	2,000 MB peak	1,200 MB peak	Fewer traced dependencies
Scheduler process	50 MB	50 MB	No change (already isolated)
Prisma connection pools	80 MB	50 MB	LRU eviction on tenant clients
OS + Node.js runtime	300 MB	300 MB	Fixed baseline

Buffer/cache	767 MB	500 MB	Less file caching needed
Turbopack dev cache (.next)	2,600 MB (disk)	1,500 MB (disk)	Periodic cleanup
Total used (peak)	~6,900 MB	~4,800 MB	30% reduction
Free memory	508 MB	~2,600 MB	5x improvement

Table 11: Memory budget before and after optimization

7. Recommendations Summary

The StaySuite-HospitalityOS project is a large-scale application with 1,870 source files, 918 API routes, and 107 dependencies running on a constrained 8 GB server. The primary OOM issue has been mitigated by isolating the scheduler, but the system remains under significant memory pressure from Turbopack's compilation cache, oversized client bundles, and duplicate real-time connections.

Immediate Actions (This Week):

- Add 2 GB swap space to the server as an OOM safety net
- Consolidate useSocket and useRealtime into a single WebSocket hook
- Remove NodePolyfillPlugin from client webpack config
- Replace framer-motion animations on page.tsx with CSS transitions

Short-Term (Next Sprint):

- Lazy-load recharts components with next/dynamic
- Merge 4 context initialization fetches into /api/settings/bundle
- Add LRU eviction to db-tenant.ts PrismaClient cache
- Lazy-import bcryptjs in auth.ts

Long-Term (Next Quarter):

- Audit and remove unused packages (ldapjs, pg, xlsx if unused)
- Enable reactStrictMode and fix any detected memory leaks
- Set up periodic Turbopack cache cleanup
- Consider upgrading to a 16 GB server for production workloads
- Implement bundle analysis in CI/CD to catch future bloat